COMPETITION MASTER 02 — ESP32 Complete Smart Farm

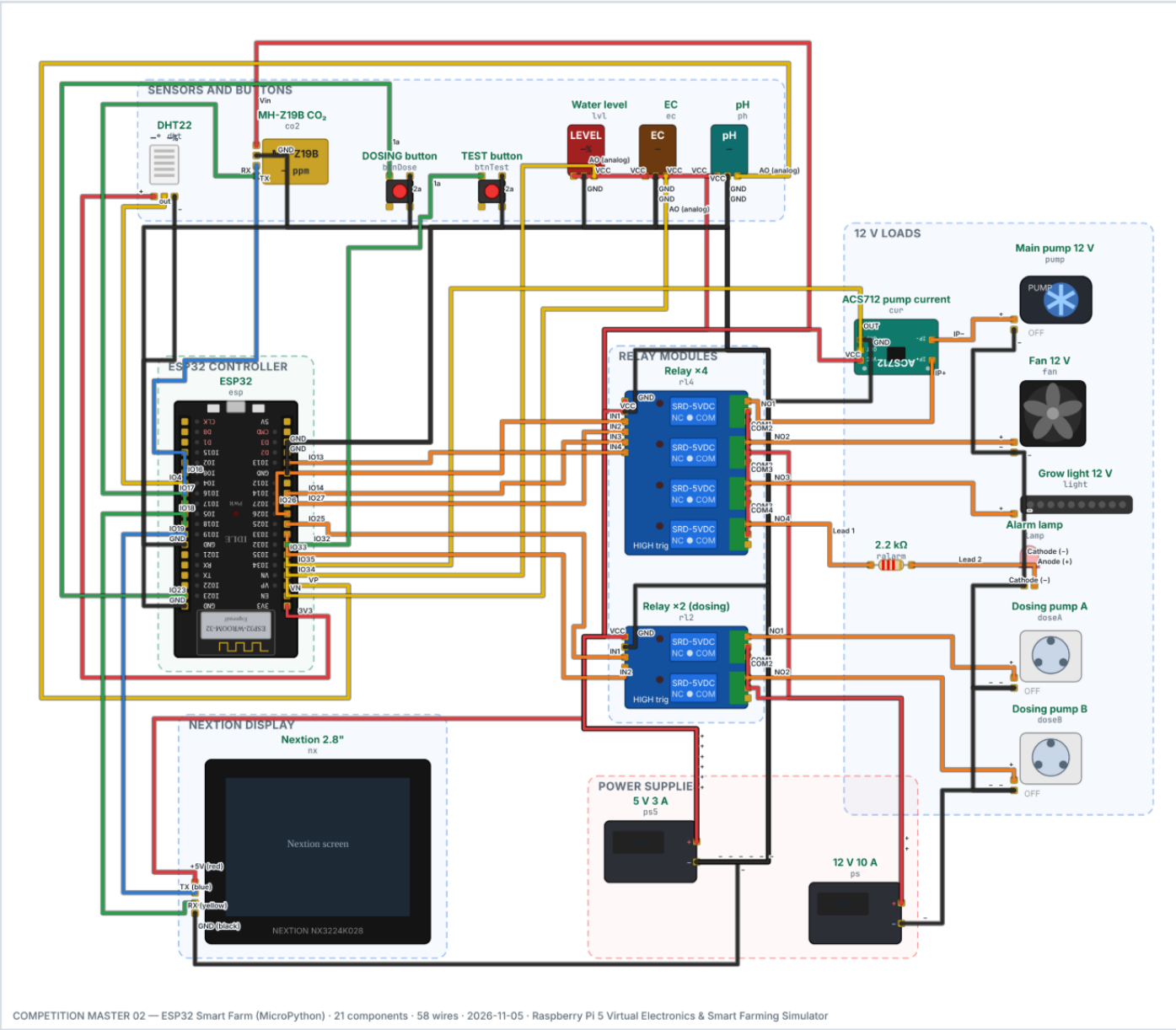
Complete study file

The same farm and the same program structure as the Raspberry Pi, written with machine.Pin / ADC / UART, with the status on the serial monitor and on a Nextion screen.

Controller	ESP32-DevKitC V4 (ESP32-WROOM-32)
Language	MicroPython 1.29 (machine, dht, time)
Program file	esp/main.py
Status display	Serial monitor (STATUS lines) + Nextion screen (Nextion tab). No web server.
Parts / wires	21 / 58
Exported	2026-11-05

DHT22 (GPIO4), MH-Z19B (UART2), pH/EC/level/ACS712 on ADC1 (GPIO36/39/34/35), Nextion (UART1), relays on GPIO26/27/14/13/25/33, buttons on GPIO23/32, 5 V and 12 V supplies.

1. Circuit diagram



21 parts · 58 wires

2. Wiring table — every wire explained

#	From	To	Type	Why	If disconnected
1	5 V 3 A · –	ESP32 · GND	Ground	common ground: the 5 V parts and the controller must share GND so their signals have the same reference	the parts on this wire lose 5 V; without the common GND wire every signal loses its reference and the readings are wrong
2	ESP32 · 3V3	DHT22 · +	Power	power for the DHT22	the DHT22 loses power — no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
3	DHT22 · out	ESP32 · IO4	Digital data	DHT22: one-wire digital data line to the ESP32	no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
4	DHT22 · –	ESP32 · GND	Ground	ground (return path) of the DHT22	the DHT22 loses power — no answer from the DHT22: temperature = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON for safety
5	5 V 3 A · +	MH-Z19B CO_2 · Vin	Power	power for the MH-Z19B CO_2 sensor	the MH-Z19B CO_2 sensor loses power — no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
6	MH-Z19B CO_2 · GND	5 V 3 A · –	Ground	ground (return path) of the MH-Z19B CO_2 sensor	the MH-Z19B CO_2 sensor loses power — no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
7	ESP32 · IO17	MH-Z19B CO_2 · RX	Digital data	MH-Z19B CO_2 sensor: UART: sensor TX → controller RX, sensor RX ← controller TX (crossed)	no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
8	MH-Z19B CO_2 · TX	ESP32 · IO16	Digital data	MH-Z19B CO_2 sensor: UART: sensor TX → controller RX, sensor RX ← controller TX (crossed)	no reply to the 0x86 command: co2 = None, the reading is lost: SENSOR ERROR alarm, alarm lamp on
9	5 V 3 A · +	pH · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
10	pH · GND	5 V 3 A · –	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
11	pH · VCC	EC · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
12	EC · GND	pH · GND	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
13	EC · VCC	Water level · VCC	Power	power for the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
14	Water level · GND	EC · GND	Ground	ground (return path) of the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
15	pH · AO (analog)	ESP32 · VP	Analog signal	pH sensor module: analog voltage: pH = 3.5 × V	pH reads wrong (near 0 → below 1): SENSOR ERROR
16	EC · AO (analog)	ESP32 · VN	Analog signal	EC sensor module: analog voltage: EC = V ÷ 0.5	EC reads wrong (near 0): OUT OF RANGE
17	Water level · AO (analog)	ESP32 · IO34	Analog signal	water-level sensor: analog voltage: 3 V = 100 %	level reads wrong (near 0 %): LOW WATER, irrigation and dosing refused
18	5 V 3 A · +	ACS712 pump current · VCC	Power	power for the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE
19	ACS712 pump current · GND	5 V 3 A · –	Ground	ground (return path) of the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE

#	From	To	Type	Why	If disconnected
20	ACS712 pump current · OUT	ESP32 · IO35	Analog signal	ACS712 current sensor: analog voltage: 2.5 V + 0.185 V per ampere of pump current	the pump current reads wrong: the next irrigation reports PUMP FAILURE
21	5 V 3 A · +	Nextion 2.8" · +5V (red)	Power	power for the Nextion screen	the Nextion screen loses power — the screen stops updating; the farm keeps working (the program only sends)
22	Nextion 2.8" · GND (black)	5 V 3 A · –	Ground	ground (return path) of the Nextion screen	the Nextion screen loses power — the screen stops updating; the farm keeps working (the program only sends)
23	ESP32 · IO18	Nextion 2.8" · RX (yellow)	Digital data	Nextion screen: UART: ESP32 TX1 → screen RX (commands); screen TX → RX1 (touch events, not used here)	the screen stops updating; the farm keeps working (the program only sends)
24	Nextion 2.8" · TX (blue)	ESP32 · IO19	Digital data	Nextion screen: UART: ESP32 TX1 → screen RX (commands); screen TX → RX1 (touch events, not used here)	the screen stops updating; the farm keeps working (the program only sends)
25	5 V 3 A · +	Relay ×4 · VCC	Power	power for the 4-channel relay module	the relay coils lose power: none of pump, fan, light, alarm lamp can run
26	Relay ×4 · GND	5 V 3 A · –	Ground	ground (return path) of the 4-channel relay module	the relay coils lose power: none of pump, fan, light, alarm lamp can run
27	ESP32 · IO26	Relay ×4 · IN1	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
28	ESP32 · IO27	Relay ×4 · IN2	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
29	ESP32 · IO14	Relay ×4 · IN3	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
30	ESP32 · IO13	Relay ×4 · IN4	Digital data	4-channel relay module: relay input: a HIGH level closes that channel	that relay no longer follows the program: the program prints pump=ON but the load stays off
31	5 V 3 A · +	Relay ×2 (dosing) · VCC	Power	power for the 2-channel relay module (dosing)	both dosing relays lose power: no dosing at all
32	Relay ×2 (dosing) · GND	5 V 3 A · –	Ground	ground (return path) of the 2-channel relay module (dosing)	both dosing relays lose power: no dosing at all
33	ESP32 · IO25	Relay ×2 (dosing) · IN1	Digital data	2-channel relay module (dosing): relay input: a HIGH level closes that channel	that dosing pump never runs although the program prints DOSING …
34	ESP32 · IO33	Relay ×2 (dosing) · IN2	Digital data	2-channel relay module (dosing): relay input: a HIGH level closes that channel	that dosing pump never runs although the program prints DOSING …
35	ACS712 pump current · IP−	Main pump 12 V · +	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
36	2.2 kΩ · Lead 2	Alarm lamp · Anode (+)	Digital data	alarm lamp: 12 V lamp (LED + 2.2 kΩ)	alarms are still printed but the lamp never lights
37	12 V 10 A · +	Relay ×4 · COM1	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
38	Relay ×4 · NO1	ACS712 pump current · IP+	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
39	Relay ×4 · COM1	Relay ×4 · COM2	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
40	Relay ×4 · NO2	Fan 12 V · +	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
41	Relay ×4 · COM2	Relay ×4 · COM3	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply –	the light stays off although the program prints light=ON

#	From	To	Type	Why	If disconnected
42	Relay ×4 · NO3	Grow light 12 V · +	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply −	the light stays off although the program prints light=ON
43	Relay ×4 · COM3	Relay ×4 · COM4	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply −	alarms are still printed but the lamp never lights
44	Relay ×4 · NO4	2.2 kΩ · Lead 1	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply −	alarms are still printed but the lamp never lights
45	12 V 10 A · +	Relay ×2 (dosing) · COM1	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
46	Relay ×2 (dosing) · NO1	Dosing pump A · +	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
47	Relay ×2 (dosing) · COM1	Relay ×2 (dosing) · COM2	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply −	dose B is not delivered although DOSING B is printed
48	Relay ×2 (dosing) · NO2	Dosing pump B · +	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply −	dose B is not delivered although DOSING B is printed
49	Main pump 12 V · −	Fan 12 V · −	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply −	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
50	Fan 12 V · −	Grow light 12 V · −	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply −	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
51	Grow light 12 V · −	Alarm lamp · Cathode (−)	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply −	the light stays off although the program prints light=ON
52	Alarm lamp · Cathode (−)	Dosing pump A · −	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
53	Dosing pump A · −	Dosing pump B · −	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply −	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
54	Dosing pump B · −	12 V 10 A · −	Ground	ground (return path) of the dosing pump B	the dosing pump B loses power — dose B is not delivered although DOSING B is printed
55	ESP32 · IO23	DOSING button · 1a	Digital data	DOSING button: input with internal pull-up: pressed = connected to GND = 0	the button does nothing (the pull-up keeps the input at "not pressed")
56	DOSING button · 2a	ESP32 · GND	Ground	ground (return path) of the DOSING button	the DOSING button loses power — the button does nothing (the pull-up keeps the input at "not pressed")
57	ESP32 · IO32	TEST button · 1a	Digital data	TEST button: input with internal pull-up: pressed = connected to GND = 0	the button does nothing (the pull-up keeps the input at "not pressed")
58	TEST button · 2a	ESP32 · GND	Ground	ground (return path) of the TEST button	the TEST button loses power — the button does nothing (the pull-up keeps the input at "not pressed")

3. Parts list

#	Component	Model / reference	Qty	Key properties	Voltage	Simulation level	IDs	Assembly notes
1	ESP32-DevKitC V4 (ESP32-WROOM-32)	ESP32-DevKitC V4 · Espressif	1	Power source: Micro-USB from the PC (5 V)	5 V USB or 5V pin (on-board 3.3 V regulator); GPIO 3.3 V, not 5 V tolerant	Functional Model	esp	
2	DC power supply	Bench / adapter DC supply	1	Voltage: 5 V; Current limit: 3 A	—	Fully Simulated	ps5	
3	DHT22 temperature/humidity	DHT22 / AM2302 module · Aosong	1	Module with pull-up resistor: yes	3.3–6 V	Fully Simulated	dht	
4	MH-Z19B CO_2 sensor	MH-Z19B · Winsen	1	—	4.5–5.5 V supply, 3.3 V UART	Fully Simulated	co2	
5	pH sensor (analog)	Analog pH probe + board (SEN0161-style)	1	—	3.3–5 V	Functional Model	ph	
6	EC conductivity sensor (analog)	Analog EC probe + board	1	—	3.3–5 V	Functional Model	ec	
7	Water level sensor (analog)	Analog water-level sensor	1	—	3.3–5 V	Functional Model	lvl	
8	ACS712 current sensor (5 A)	ACS712ELCTR-05B · Allegro	1	Version: 5 A (185 mV/A)	5 V supply	Fully Simulated	cur	
9	Nextion Enhanced 2.8" (NX3224K028)	NX3224K028 · ITEAD	1	Baud rate (bauds): 9600	4.75–7 V supply, 3.3 V TX (5 V-tolerant RX)	Fully Simulated	nx	
10	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 4; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	Fully Simulated	rl4	
11	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 2; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	Fully Simulated	rl2	
12	DC power supply	Bench / adapter DC supply	1	Voltage: 12 V; Current limit: 10 A	—	Fully Simulated	ps	
13	Water pump (12 V DC)	12 V DC pump	1	Rated power: 6 W	12 V	Functional Model	pump	
14	Cooling fan (DC)	12 V DC fan	1	Rated power: 2.4 W	12 V	Functional Model	fan	
15	LED grow light	12 V LED grow bar	1	Rated power: 36 W	12 V	Functional Model	light	
16	Resistor	¼ W carbon-film resistor	1	Resistance: 2.2 kΩ	—	Fully Simulated	ralarm	
17	LED	5 mm LED	1	Colour: red	Vf ≈ 1.9–2.9 V, 20 mA max	Fully Simulated	lamp	
18	Peristaltic dosing pump	12 V peristaltic pump	2	Rated power: 3.6 W	12 V	Functional Model	doseA, doseB	
19	Push button	6×6 mm tactile switch	2	—	—	Fully Simulated	btnDose, btnTest	

#	Component	Model / reference	Qty	Key properties	Voltage	Simulation level	IDs	Assembly notes
20	Jumper wires	—	58	18× black, 15× red, 5× yellow, 4× green, 2× blue, 14× orange	—	—		Use male–female jumpers for the Pi header; keep power wires short.

4. Settings (constants at the top of the program)

Name	Value	Meaning
MAX_TEMP	30	فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP	27	تحت 27 درجة: إيقاف المروحة
MIN_WATER	20	أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS	5	مدة الري بالثواني
IRRIGATION_HOURS	[7, 12, 17]	ساعات الري في اليوم
LIGHT_ON_HOUR	6	ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR	20	ساعة إطفاء الإضاءة
DOSE_A_ML	10	جرعة السماد A بالمليلتر
DOSE_B_ML	5	جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC	2	تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN	5.5	أقل pH مقبول
PH_MAX	6.5	أعلى pH مقبول
EC_MIN	1.0	أقل EC مقبول (mS/cm)
EC_MAX	2.5	أعلى EC مقبول
MIN_PUMP_AMPS	0.2	أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS	2	مدة تشغيل كل جهاز في وضع الاختبار

Training values you may change; they are not certified settings for every crop. Dose accuracy depends on the calibration of the real pumps.

5. The program in 9 sections

1 · Setup and wiring

Same settings as the Pi. machine.Pin, ADC and UART take the ESP32 GPIO numbers; the analog sensors use ADC1 pins (36, 39, 34, 35) which keep working with Wi-Fi.

- Change a threshold here and the whole program follows it.
- GPIO34–39 are input-only; never use them as outputs.

esp/main.py

```
---------- Raspberry Pi الإعدادات: نفس قيم مشروع (1 ---------- #  7
تشغيل المروحة: فوق 30 درجة: #                MAX_TEMP = 30  8
إيقاف المروحة: تحت 27 درجة: #             FAN_OFF_TEMP = 27  9
لتشغيل المضخة (%) أقل مستوى ماء مسموح #                MIN_WATER = 20  10
مدة الري بالثواني #              PUMP_SECONDS = 5  11
ساعات الري في اليوم #  IRRIGATION_HOURS = [7, 12, 17]  12
ساعة تشغيل الإضاءة #             LIGHT_ON_HOUR = 6  13
ساعة إطفاء الإضاءة #           LIGHT_OFF_HOUR = 20  14
جرعة السماد A بالمليلتر #                DOSE_A_ML = 10  15
جرعة السماد B بالمليلتر #                DOSE_B_ML = 5  16
تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي) #           FLOW_ML_PER_SEC = 2  17
أقل pH مقبول #                  PH_MIN = 5.5  18
أعلى pH مقبول #                  PH_MAX = 6.5  19
أقل EC مقبول (mS/cm) #                  EC_MIN = 1.0  20
أعلى EC مقبول #                  EC_MAX = 2.5  21
أعلى CO2 مقبول (ppm) #                CO2_MAX = 1500  22
أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير) #           MIN_PUMP_AMPS = 0.2  23
مدة تشغيل كل جهاز في وضع الاختبار #              TEST_SECONDS = 2  24
 25
 26  # >>> PINS
---------- التوصيلات: كل جهاز على رجله (2 ---------- #  27
GPIO4 على البيانات :dht_sensor = dht.DHT22(Pin(4))                      # DHT22  28
UART2 (TX=17، RX=16) على co2_port = UART(2, baudrate=9600, tx=17, rx=16, timeout=300)  # MH-Z19B  29
UART1 (TX=18، RX=19) على Nextion شاشة #       screen = UART(1, baudrate=9600, tx=18, rx=19)  30
GPIO36 (ADC1 على ph_adc = ADC(Pin(36), atten=ADC.ATTN_11DB)          # pH  31
↪ (تقريباً 3.3 فولت، حتى Wi-Fi يعمل مع
```

```
GPIO39 (ADC1) على ec_adc = ADC(Pin(39), atten=ADC.ATTN_11DB)          # EC  32
GPIO34 (ADC1) على مستوى الماء #       level_adc = ADC(Pin(34), atten=ADC.ATTN_11DB)  33
GPIO35 (ADC1) على ACS712 تيار المضخة #     current_adc = ADC(Pin(35), atten=ADC.ATTN_11DB)  34
(يبدأ مطفأً) GPIO26 على مرحّل مضخة الري #                    pump = Pin(26, Pin.OUT, value=0)  35
GPIO27 على مرحّل المروحة #                    fan = Pin(27, Pin.OUT, value=0)  36
GPIO14 على مرحّل الإضاءة #                   light = Pin(14, Pin.OUT, value=0)  37
GPIO13 على مرحّل مصباح الإنذار #              alarm_lamp = Pin(13, Pin.OUT, value=0)  38
GPIO25 على A مرحّل مضخة الجرعات #                  dose_a = Pin(25, Pin.OUT, value=0)  39
GPIO33 على B مرحّل مضخة الجرعات #                  dose_b = Pin(33, Pin.OUT, value=0)  40
(GND إلى، مقاومة سحب داخلية) GPIO23 على زر الجرعات #        dosing_button = Pin(23, Pin.IN, Pin.PULL_UP)  41
(GND إلى) GPIO32 على زر الاختبار #          test_button = Pin(32, Pin.IN, Pin.PULL_UP)  42
 43  # <<< PINS
```

2 · Reading the sensors

One function reads everything and stores it in variables. A missing reading becomes None/null, so the alarms can see it. Analog sensors give volts; one formula per sensor turns volts into pH, EC, % and amperes.

- pH = 3.5 × V · EC = V ÷ 0.5 · level = V ÷ 3 × 100 · amps = (V − 2.5) ÷ 0.185 (the simulator modules; calibrate real probes).
- DHT22 needs 2 s between readings: the program reads it at most every 2 s.

esp/main.py

```
UART عبر MH-Z19B من CO2 قراءة #                                    :()def read_co2  65
(9 بايت) أمر طلب القراءة #  co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0x79]))      66
(أو None بعد المهلة) ننتظر رداً من 9 بايت #                        reply = co2_port.read(9)      67
مفصول الحساس = رد لا #           :if reply is None or len(reply) != 9      68
تعني: لا توجد قراءة return None                                  # None          69
ppm التركيز = 3 و 2 البايت #                return reply[2] * 256 + reply[3]      70
 73  # >>> SENSORS
قراءة كل الحساسات #                                  :()def read_sensors  74
هذه سنغيّر #  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time      75
↪ المتغيرات العامة
فقط ثانيتين كل جديدة قراءة :if time.time() - last_dht_time >= 2:            # DHT22      76
القراءة هذا وقت نحفظ #                ()last_dht_time = time.time          77
القراءة نحاول #                                        :try          78
أولاً القياس نطلب MicroPython في #                    ()dht_sensor.measure              79
الحرارة C° #  ()temperature = dht_sensor.temperature              80
الرطوبة % #       ()humidity = dht_sensor.humidity              81
(الحساس مفصول) القراءة فشل #                              :except OSError          82
حرارة قراءة توجد لا #                      temperature = None              83
رطوبة ولا #                        humidity = None              84
(أو None) co2 = read_co2()                               # CO2 ppm      85
الجهد × Raspberry Pi: pH = 3.5 نفس معادلة #               ph = round(volts(ph_adc) * 3.5, 2)      86
الجهد ÷ 0.5 = ec = round(volts(ec_adc) / 0.5, 2)              # EC      87
فولت = %100 water_level = round(volts(level_adc) / 3.0 * 100)  # 3      88
أمبير صفر = فولت pump_amps = round((volts(current_adc) - 2.5) / 0.185, 2)  # ACS712: 2.5      89
 90  # <<< SENSORS
```

3 · Driving the outputs

Same idea as the Pi: Pin.on()/off() drive the relay inputs (3.3 V logic, active-HIGH module); the relays switch the 12 V loads.

- Loads: pump, fan, light and dosing pumps on the 12 V supply, never on the controller pins.
- The alarm lamp is a 12 V lamp on its own relay channel.

esp/main.py

```
(يبدأ مطفأً) GPIO26 على مرحّل مضخة الري #                    pump = Pin(26, Pin.OUT, value=0)  35
GPIO27 على مرحّل المروحة #                    fan = Pin(27, Pin.OUT, value=0)  36
GPIO14 على مرحّل الإضاءة #                   light = Pin(14, Pin.OUT, value=0)  37
GPIO13 على مرحّل مصباح الإنذار #              alarm_lamp = Pin(13, Pin.OUT, value=0)  38
GPIO25 على A مرحّل مضخة الجرعات #                  dose_a = Pin(25, Pin.OUT, value=0)  39
GPIO33 على B مرحّل مضخة الجرعات #                  dose_b = Pin(33, Pin.OUT, value=0)  40
لعرض حالة جهاز "OFF" أو "def on_off(device):                                  # "ON  206
0 أو (يعمل) 1 تعطي ()if device.value() == 1:                              # Pin.value      207
يعمل #                                  "return "ON          208
متوقف #                                      "return "OFF      209
```

4 · Conditions and protection

water_ok() is the one place that decides "is there enough water?". Irrigation checks it before and during every second, and checks the pump current: a pump that runs dry or is not connected draws no current, so the program stops it and remembers the failure.

- A level above 100 % is treated as an invalid reading, not as "full".
- The pump is always switched off at the end, whatever stopped the irrigation.

esp/main.py

```
def water_ok():  # 93
    return water_level >= MIN_WATER and water_level <= 100  # 94 قراءة غير صالحة = أكثر من 100%
# >>> IRRIGATION  101
def irrigate():  # 102 ري لمدة PUMP_SECONDS مع الحماية
    global pump_failed  # 103 قد نغيّر هذا المتغير العام
    print("IRRIGATION start")  # 104 رسالة بداية الري
    for second in range(PUMP_SECONDS):  # 105 ثانية بعد ثانية
        read_sensors()  # 106 قراءة جديدة قبل كل ثانية
        if not water_ok():  # 107 الماء نقص أثناء الري
            # print("IRRIGATION stopped: LOW WATER")  108 نخبر بالسبب
            break  # 109 نخرج من الحلقة فوراً
        pump.on()  # 110 تشغيل المضخة
        time.sleep(1)  # 111 انتظار ثانية
        read_sensors()  # 112 نقرأ تيار المضخة الآن
        if pump_amps < MIN_PUMP_AMPS:  # 113 المضخة لا تسحب تياراً = لا تعمل
            pump_failed = True  # 114 نتذكر العطل (يظهر الإنذار)
            print("PUMP FAILURE: no current")  # 115 رسالة العطل
            break  # 116 نتوقف عن الري
    pump.off()  # 117 إيقاف المضخة دائماً في النهاية
    print("IRRIGATION end")  # 118 رسالة نهاية الري
# <<< IRRIGATION  119
```

5 · AUTO mode

Every second: the fan follows the temperature with two thresholds (ON above 30 °C, OFF below 27 °C — between them it keeps its state, so it does not click on and off), the light follows the clock (06:00–20:00), and irrigation starts once at 07:00, 12:00 and 17:00 if there is water.

- No temperature reading → the fan runs (the safe side for the plants).
- last_irrigation_hour prevents watering again and again during the same hour.

esp/main.py

```
def auto_control():  # 156 الوضع التلقائي
    global last_irrigation_hour  # 157 سنغيّر هذا المتغير العام
    hour = time.localtime()[3]  # 158 الساعة الحالية: في MicroPython هي العنصر رقم 3
# >>> FAN  159
    if temperature is None:  # 160 لا قراءة حرارة: نشغّل المروحة للأمان
        fan.on()  # 161 تشغيل المروحة
    elif temperature > MAX_TEMP:  # 162 حار جداً
        fan.on()  # 163 تشغيل المروحة
    elif temperature < FAN_OFF_TEMP:  # 164 برد بما يكفي
        fan.off()  # 165 إيقاف المروحة (بين 27 و 30 تبقى كما هي)
# <<< FAN  166
# >>> LIGHT  167
    if LIGHT_ON_HOUR <= hour < LIGHT_OFF_HOUR:  # 168 داخل ساعات الإضاءة
        light.on()  # 169 تشغيل الإضاءة
    else:  # 170 خارج ساعات الإضاءة
        light.off()  # 171 إطفاء الإضاءة
# <<< LIGHT  172
    if hour in IRRIGATION_HOURS and hour != last_irrigation_hour:  # 173 حان وقت ري جديد
        last_irrigation_hour = hour  # 174 نتذكر الساعة حتى لا نكرر الري
        if water_ok():  # 175 يوجد ماء؟
            irrigate()  # 176 نروي
        else:  # 177 لا يوجد ماء
            # print("IRRIGATION skipped: LOW WATER")  178 نتخطى الري
```

6 · DOSING mode

Check the water, switch B off, run A for (dose ÷ flow) seconds, switch A off, run B, switch B off, print DOSING DONE. A and B are never on together. The volumes are only as accurate as the flow calibration of the real pumps

(here 2 ml/s).

- Started by the DOSING button (or /mode/DOSING on the Pi web page).
- The Smart Farm counts the millilitres really pumped, so you can check the doses.

esp/main.py

```
122  # >>> DOSING
(لا تعملان معاً أبداً) B ثم جرعة A جرعة #                                    :()def run_dosing  123
لا تسميد بدون ماء #                                :()if not water_ok      124
نخبر بالسبب #        print("DOSING cancelled: LOW WATER")          125
نخرج من الدالة #                                      return          126
A متوقفة قبل تشغيل B تأكد أن #                                    ()dose_b.off      127
A تشغيل #                                    ()dose_a.on      128
A رسالة الجرعة #              print("DOSING A", DOSE_A_ML, "ml")      129
(ثوانٍ 5 = 2 ÷ 10) الزمن = الكمية ÷ التدفق #         time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)      130
A إيقاف #                                   ()dose_a.off      131
B ثم تشغيل #                                      ()dose_b.on      132
B رسالة الجرعة #              print("DOSING B", DOSE_B_ML, "ml")      133
ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)          # 5 ÷ 2 = 2.5      134
B إيقاف #                                   ()dose_b.off      135
انتهت الجرعات #                             print("DOSING DONE")      136
137  # <<< DOSING
```

7 · TEST mode

Each output runs for 2 s in turn: fan, light, dose A, dose B, then the pump — and the pump only if there is water. The protection stays active in test mode.

- Use it after wiring a real farm: every relay should click once and every load should run.
- Dose A and dose B run one after the other, never together.

esp/main.py

```
تجربة كل جهاز لثانيتين #                                    :()def run_test  140
قائمة #  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]      141
↪ الأجهزة
اسم الجهاز الذي نختبره #                        print("TEST", name)          142
تشغيله #                          ()device.on          143
ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          144
إيقافه #                               ()device.off          145
(المضخة فقط إذا يوجد ماء (الحماية لا تُلغى #                                    :()if water_ok      146
نختبر المضخة #                             print("TEST PUMP")          147
تشغيلها #                                  ()pump.on          148
ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          149
إيقافها #                                  ()pump.off          150
لا يوجد ماء كافٍ #                                          :else      151
لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")          152
انتهى الاختبار #                               print("TEST DONE")      153
```

8 · Alarms

One if/elif chain picks the most important alarm: SENSOR ERROR, LOW WATER, PUMP FAILURE, HIGH TEMPERATURE, OUT OF RANGE, or NONE. The alarm lamp is on for any alarm, and the alarm is printed only when it changes.

- The order of the elif lines is the priority: a missing sensor matters more than a high temperature.
- PUMP FAILURE stays until the program restarts (the pump must be checked by a person).

esp/main.py

```
181  # >>> ALARMS
اختيار أهم إنذار وتشغيل المصباح #                                    :()def check_alarms  182
سنغيّر هذا المتغير العام #                                  global alarm      183
نحفظ الإنذار السابق #                                old = alarm      184
قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100      185
خطأ حساس #                      "alarm = "SENSOR ERROR          186
الماء قليل #                             :()elif not water_ok      187
نقص الماء #                         "alarm = "LOW WATER          188
المضخة لم تسحب تياراً #                                 :elif pump_failed      189
عطل المضخة #                       "alarm = "PUMP FAILURE          190
حرارة عالية #                      :elif temperature > MAX_TEMP      191
ارتفاع الحرارة #                   "alarm = "HIGH TEMPERATURE          192
```

```
خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX      193
قيمة خارج المجال #                      "alarm = "OUT OF RANGE          194
كل شيء طبيعي #                                    :else      195
لا يوجد إنذار #                                "alarm = "NONE          196
لا إنذار #                                :"if alarm == "NONE      197
إطفاء مصباح الإنذار #                              ()alarm_lamp.off          198
يوجد إنذار #                                      :else      199
تشغيل مصباح الإنذار #                              ()alarm_lamp.on          200
نطبع الإنذار عند تغيّره فقط #                                 :if alarm != old      201
رسالة الإنذار #                        print("ALARM", alarm)          202
203  # <<< ALARMS
```

9 · Display and monitoring

Every 5 s a STATUS line goes to the serial monitor and the same values to the Nextion screen (tTemp.txt="..." + three 0xFF). There is no web server on this ESP32 program; on real hardware one could be added with Wi-Fi later.

- STATUS mode=... temp=... ... alarm=... day=... is the same line on the three controllers.
- The status panel of this page reads that line from the running program.

esp/main.py

```
Nextion كتابة نص في عنصر على شاشة #                                 :def show(name, text)  212
"tTemp.txt="24.5 :مثال #  screen.write(name + '.txt="' + str(text) + '"')      213
0xFF ينتهي بثلاثة Nextion كل أمر #         screen.write(bytes([0xFF, 0xFF, 0xFF]))      214
215
216
وعلى الشاشة Serial الحالة على #                                    :()def show_status  217
218      print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
↪ الوضع والهواء
الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"            219
↪ والمضخة
220            "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
↪ الباقي
الحرارة على الشاشة #                      show("tTemp", temperature)      221
الرطوبة #                          show("tHum", humidity)      222
223      show("tCO2", co2)                                 # CO2
224      show("tPH", ph)                                   # pH
225      show("tEC", ec)                                   # EC
مستوى الماء #                     show("tLevel", water_level)      226
الوضع #                             show("tMode", mode)      227
الإنذار #                           show("tAlarm", alarm)      228
يوم النمو #                       show("tDay", growth_day())      229
---------- الحلقة الرئيسية (4 ---------- #  232
Serial Monitor رسالة البداية على #                 print("Smart farm started (ESP32)")  233
عدّاد الدورات #                                          loop = 0  234
تكرار إلى الأبد #                                       :while True  235
قراءة الحساسات (read_sensors()                                  # 1      236
(GND موصول إلى) 0 الزر المضغوط يقرأ (if dosing_button.value() == 0:                  # 2      237
نختار وضع الجرعات #                                "mode = "DOSING          238
زر الاختبار مضغوط؟ #                     :if test_button.value() == 0      239
نختار وضع الاختبار #                               "mode = "TEST          240
تنفيذ الوضع (if mode == "DOSING":                                # 3      241
B ثم A الجرعات #                                   ()run_dosing          242
ثم العودة إلى التلقائي #                                 "mode = "AUTO          243
وضع الاختبار #                            :"elif mode == "TEST      244
تجربة الأجهزة #                                    ()run_test          245
ثم العودة إلى التلقائي #                                "mode = "AUTO          246
AUTO الوضع #                                            :else      247
المروحة والإضاءة والري #                                ()auto_control          248
الإنذارات (check_alarms()                                    # 4      249
الحالة كل 5 ثوانٍ (if loop % 5 == 0:                                 # 5      250
251          show_status()                                 # Serial + Nextion
الدورة التالية #                                loop = loop + 1      252
انتظار ثانية ثم التكرار (time.sleep(1)                                      # 6      253
```

6. Complete program

esp/main.py

```
# COMPETITION MASTER 02 — ESP32 Complete Smart Farm (MicroPython)
# نفس منطق مشروع Raspberry Pi تماماً — الاختلاف فقط في طريقة الوصول إلى الأرجل
import time                                          # الوقت والانتظار (sleep)
import dht                                           # حساس الحرارة والرطوبة DHT22
from machine import Pin, ADC, UART                   # الأرجل + محوّل ADC الداخلي + منافذ UART

# ---------- 1) الإعدادات: نفس قيم مشروع Raspberry Pi ----------
MAX_TEMP = 30                # فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP = 27            # تحت 27 درجة: إيقاف المروحة
MIN_WATER = 20               # أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS = 5             # مدة الري بالثواني
# IRRIGATION_HOURS = [7, 12, 17]  # ساعات الري في اليوم
LIGHT_ON_HOUR = 6            # ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR = 20          # ساعة إطفاء الإضاءة
DOSE_A_ML = 10               # جرعة السماد A بالمليلتر
DOSE_B_ML = 5                # جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC = 2          # تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN = 5.5                 # أقل pH مقبول
PH_MAX = 6.5                 # أعلى pH مقبول
EC_MIN = 1.0                 # أقل EC مقبول (mS/cm)
EC_MAX = 2.5                 # أعلى EC مقبول
CO2_MAX = 1500               # أعلى CO2 مقبول (ppm)
MIN_PUMP_AMPS = 0.2          # أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS = 2             # مدة تشغيل كل جهاز في وضع الاختبار

# >>> PINS
# ---------- 2) التوصيلات: كل جهاز على رجله ----------
dht_sensor = dht.DHT22(Pin(4))                       # DHT22 البيانات على GPIO4
co2_port = UART(2, baudrate=9600, tx=17, rx=16, timeout=300)  # MH-Z19B على UART2 (TX=17، RX=16)
screen = UART(1, baudrate=9600, tx=18, rx=19)        # شاشة Nextion على UART1 (TX=18، RX=19)
ph_adc = ADC(Pin(36), atten=ADC.ATTN_11DB)           # pH على GPIO36 (ADC1) يعمل مع Wi-Fi، حتى 3.3 فولت تقريباً
ec_adc = ADC(Pin(39), atten=ADC.ATTN_11DB)           # EC على GPIO39 (ADC1)
level_adc = ADC(Pin(34), atten=ADC.ATTN_11DB)        # مستوى الماء على GPIO34 (ADC1)
current_adc = ADC(Pin(35), atten=ADC.ATTN_11DB)      # تيار المضخة ACS712 على GPIO35 (ADC1)
pump = Pin(26, Pin.OUT, value=0)                     # مرحّل مضخة الري على GPIO26 (يبدأ مطفأ)
fan = Pin(27, Pin.OUT, value=0)                      # مرحّل المروحة على GPIO27
light = Pin(14, Pin.OUT, value=0)                    # مرحّل الإضاءة على GPIO14
alarm_lamp = Pin(13, Pin.OUT, value=0)               # مرحّل مصباح الإنذار على GPIO13
dose_a = Pin(25, Pin.OUT, value=0)                   # مرحّل مضخة الجرعات A على GPIO25
dose_b = Pin(33, Pin.OUT, value=0)                   # مرحّل مضخة الجرعات B على GPIO33
dosing_button = Pin(23, Pin.IN, Pin.PULL_UP)         # زر الجرعات على GPIO23 (إلى GND، مقاومة سحب داخلية)
test_button = Pin(32, Pin.IN, Pin.PULL_UP)           # زر الاختبار على GPIO32 (إلى GND)
# <<< PINS

# ---------- 3) حالة النظام ----------
mode = "AUTO"                # الوضع الحالي: AUTO أو DOSING أو TEST
alarm = "NONE"               # الإنذار الحالي
pump_failed = False          # يصبح True إذا شغّلنا المضخة ولم يمر تيار
temperature = None           # آخر قراءة حرارة
humidity = None              # آخر قراءة رطوبة
co2 = None                   # آخر قراءة CO2
ph = 0                       # آخر قراءة pH
ec = 0                       # آخر قراءة EC
water_level = 0              # مستوى الماء %
pump_amps = 0                # تيار المضخة بالأمبير
last_irrigation_hour = -1    # آخر ساعة تم فيها الري (حتى لا نكرره)
start_time = time.time()     # وقت بدء التشغيل لحساب يوم النمو
last_dht_time = 0            # وقت آخر قراءة DHT22 (يحتاج ثانيتين بين القراءات)


def volts(adc):                                      # جهد مدخل ADC بالفولت
    return adc.read_uv() / 1000000                   # read_uv تعطي ميكروفولت معايَراً


def read_co2():                                      # قراءة CO2 من MH-Z19B عبر UART
    co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))  # أمر طلب القراءة (9 بايت)
    reply = co2_port.read(9)                         # ننتظر رداً من 9 بايت (أو None بعد المهلة)
    if reply is None or len(reply) != 9:             # لا رد = الحساس مفصول
        return None                                  # None تعني: لا توجد قراءة
    return reply[2] * 256 + reply[3]                 # البايت 2 و 3 = التركيز ppm

```

```
72
73 # >>> SENSORS
74 قراءة كل الحساسات # :()def read_sensors
75 سنغيّر هذه المتغيرات # global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time
↪ العامة
76 قراءة جديدة كل ثانيتين فقط :if time.time() - last_dht_time >= 2: # DHT22
77 نحفظ وقت هذه القراءة # ()last_dht_time = time.time
78 نحاول القراءة # :try
79 نطلب القياس أولاً MicroPython في # ()dht_sensor.measure
80 C° الحرارة # ()temperature = dht_sensor.temperature
81 % الرطوبة # ()humidity = dht_sensor.humidity
82 (الحساس مفصول) فشل القراءة # :except OSError
83 لا توجد قراءة حرارة # temperature = None
84 ولا رطوبة # humidity = None
85 (None أو) co2 = read_co2() # CO2 ppm
86 الجهد × Raspberry Pi: pH = 3.5 نفس معادلة # ph = round(volts(ph_adc) * 3.5, 2)
87 0.5 ÷ الجهد = ec = round(volts(ec_adc) / 0.5, 2) # EC
88 %100 = فولت water_level = round(volts(level_adc) / 3.0 * 100) # 3
89 صفر أمبير = فولت pump_amps = round((volts(current_adc) - 2.5) / 0.185, 2) # ACS712: 2.5
90 # <<< SENSORS
91
92
93 هل يوجد ماء كافٍ؟ # :()def water_ok
94 قراءة غير صالحة = %100 أكثر من # return water_level >= MIN_WATER and water_level <= 100
95
96
97 يوم النمو منذ بدء التشغيل # :()def growth_day
98 يوم واحد = ثانية return int((time.time() - start_time) / 86400) + 1 # 86400
99
100
101 # >>> IRRIGATION
102 مع الحماية PUMP_SECONDS ري لمدة # :()def irrigate
103 قد نغيّر هذا المتغير العام # global pump_failed
104 رسالة بداية الري # print("IRRIGATION start")
105 ثانية بعد ثانية # :for second in range(PUMP_SECONDS)
106 قراءة جديدة قبل كل ثانية # ()read_sensors
107 الماء نقص أثناء الري # :()if not water_ok
108 نخبر بالسبب # print("IRRIGATION stopped: LOW WATER")
109 نخرج من الحلقة فوراً # break
110 تشغيل المضخة # ()pump.on
111 انتظار ثانية # time.sleep(1)
112 نقرأ تيار المضخة الآن # ()read_sensors
113 لا تعمل = المضخة لا تسحب تياراً # :if pump_amps < MIN_PUMP_AMPS
114 (يظهر الإنذار) نتذكر العطل # pump_failed = True
115 رسالة العطل # print("PUMP FAILURE: no current")
116 نتوقف عن الري # break
117 إيقاف المضخة دائماً في النهاية # ()pump.off
118 رسالة نهاية الري # print("IRRIGATION end")
119 # <<< IRRIGATION
120
121
122 # >>> DOSING
123 (لا تعملان معاً أبداً) B ثم جرعة A جرعة # :()def run_dosing
124 لا تسميد بدون ماء # :()if not water_ok
125 نخبر بالسبب # print("DOSING cancelled: LOW WATER")
126 نخرج من الدالة # return
127 A متوقفة قبل تشغيل B تأكد أن # ()dose_b.off
128 A تشغيل # ()dose_a.on
129 A رسالة الجرعة # print("DOSING A", DOSE_A_ML, "ml")
130 (ثوانٍ 5 = 2 ÷ 10) الزمن = الكمية ÷ التدفق # time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)
131 A إيقاف # ()dose_a.off
132 B ثم تشغيل # ()dose_b.on
133 B رسالة الجرعة # print("DOSING B", DOSE_B_ML, "ml")
134 ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC) # 5 ÷ 2 = 2.5
135 B إيقاف # ()dose_b.off
136 انتهت الجرعات # print("DOSING DONE")
137 # <<< DOSING
138
139
140 تجربة كل جهاز لثانيتين # :()def run_test
141 قائمة # :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]
↪ الأجهزة
142 اسم الجهاز الذي نختبره # print("TEST", name)
143 تشغيله # ()device.on
144 ننتظر ثانيتين # time.sleep(TEST_SECONDS)
145 إيقافه # ()device.off
146 (لا تُلغى) المضخة فقط إذا يوجد ماء (الحماية # :()if water_ok
```

```
147 نختبر المضخة #                          print("TEST PUMP")
148 تشغيلها #                                  ()pump.on
149 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)
150 إيقافها #                                 ()pump.off
151 لا يوجد ماء كافٍ #                                    :else
152 لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")
153 انتهى الاختبار #                           print("TEST DONE")
154
155
156 الوضع التلقائي #                              :()def auto_control
157 سنغيّر هذا المتغير العام #                  global last_irrigation_hour
158 3 هي العنصر رقم MicroPython في الساعة الحالية: #                    hour = time.localtime()[3]
159     # >>> FAN
160 لا قراءة حرارة: نشغّل المروحة للأمان #                       :if temperature is None
161 تشغيل المروحة #                               ()fan.on
162 حار جداً #                    :elif temperature > MAX_TEMP
163 تشغيل المروحة #                               ()fan.on
164 برد بما يكفي #               :elif temperature < FAN_OFF_TEMP
165 (بين 27 و 30 تبقى كما هي) إيقاف المروحة #                                ()fan.off
166     # <<< FAN
167     # >>> LIGHT
168 داخل ساعات الإضاءة #  :if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR
169 تشغيل الإضاءة #                                ()light.on
170 خارج ساعات الإضاءة #                                  :else
171 إطفاء الإضاءة #                               ()light.off
172     # <<< LIGHT
173 حان وقت ري جديد #  :if hour in IRRIGATION_HOURS and hour != last_irrigation_hour
174 نتذكر الساعة حتى لا نكرر الري #               last_irrigation_hour = hour
175 يوجد ماء؟ #                           :()if water_ok
176 نروي #                             ()irrigate
177 لا يوجد ماء #                                 :else
178 نتخطى الري #  print("IRRIGATION skipped: LOW WATER")
179
180
181 # >>> ALARMS
182 اختيار أهم إنذار وتشغيل المصباح #                          :()def check_alarms
183 سنغيّر هذا المتغير العام #                                global alarm
184 نحفظ الإنذار السابق #                                  old = alarm
185 قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100
186 خطأ حساس #                      "alarm = "SENSOR ERROR
187 الماء قليل #                             :()elif not water_ok
188 نقص الماء #                         "alarm = "LOW WATER
189 المضخة لم تسحب تياراً #                               :elif pump_failed
190 عطل المضخة #                     "alarm = "PUMP FAILURE
191 حرارة عالية #                    :elif temperature > MAX_TEMP
192 ارتفاع الحرارة #                 "alarm = "HIGH TEMPERATURE
193 خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX
194 قيمة خارج المجال #                      "alarm = "OUT OF RANGE
195 كل شيء طبيعي #                                      :else
196 لا يوجد إنذار #                            "alarm = "NONE
197 لا إنذار #                               :"if alarm == "NONE
198 إطفاء مصباح الإنذار #                              ()alarm_lamp.off
199 يوجد إنذار #                                      :else
200 تشغيل مصباح الإنذار #                              ()alarm_lamp.on
201 نطبع الإنذار عند تغيّره فقط #                                :if alarm != old
202 رسالة الإنذار #                       print("ALARM", alarm)
203 # <<< ALARMS
204
205
206 لعرض حالة جهاز "OFF" أو "def on_off(device):                                 # "ON
207 0 تعطي 1 (يعمل) أو ()if device.value() == 1:                          # Pin.value
208 يعمل #                                  "return "ON
209 متوقف #                                      "return "OFF
210
211
212 Nextion كتابة نص في عنصر على شاشة #                                :def show(name, text)
213 "tTemp.txt="24.5 :مثال #  screen.write(name + '.txt="' + str(text) + '"')
214 0xFF ينتهي بثلاثة Nextion كل أمر #         screen.write(bytes([0xFF, 0xFF, 0xFF]))
215
216
217 وعلى الشاشة Serial الحالة على #                                  :()def show_status
218     print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
 ↪ الوضع والهواء
219 الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"
 ↪ والمضخة
220           "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
 ↪ الباقي
```

```
                    show("tTemp", temperature)  # الحرارة على الشاشة
                  show("tHum", humidity)  # الرطوبة
    show("tCO2", co2)                   # CO2
    show("tPH", ph)                     # pH
    show("tEC", ec)                     # EC
 show("tLevel", water_level)  # مستوى الماء
   show("tMode", mode)  # الوضع
   show("tAlarm", alarm)  # الإنذار
 show("tDay", growth_day())  # يوم النمو


---------- الحلقة الرئيسية (4 ---------- #
        print("Smart farm started (ESP32)")  # رسالة البداية على Serial Monitor
               loop = 0  # عدّاد الدورات
              :while True  # تكرار إلى الأبد
                  # 1  قراءة الحساسات (read_sensors()
    # 2  الزر المضغوط يقرأ 0 (موصول إلى GND) (if dosing_button.value() == 0:
            "mode = "DOSING  # نختار وضع الجرعات
       :if test_button.value() == 0  # زر الاختبار مضغوط؟
                 "mode = "TEST  # نختار وضع الاختبار
                    # 3  تنفيذ الوضع (if mode == "DOSING":
            ()run_dosing  # الجرعات A ثم B
                   "mode = "AUTO  # ثم العودة إلى التلقائي
     :"elif mode == "TEST  # وضع الاختبار
            ()run_test  # تجربة الأجهزة
                   "mode = "AUTO  # ثم العودة إلى التلقائي
                   :else  # الوضع AUTO
                ()auto_control  # المروحة والإضاءة والري
                    # 4  الإنذارات (check_alarms()
                       # 5  الحالة كل 5 ثوانٍ (if loop % 5 == 0:
   # Serial + Nextion  show_status()
 loop = loop + 1  # الدورة التالية
                          # 6  انتظار ثانية ثم التكرار (time.sleep(1)

```

7. Test scenarios and results

Results of the scenarios run in the browser on 2026-11-05 (real runtime of the simulator).

Scenario	What it does	Expected	Result
1 · Normal	08:00, 25 °C, water 75 %: run for 12 s without touching anything.	STATUS lines with all readings, mode AUTO, light ON (06–20 h), fan and pump OFF, alarm NONE, alarm lamp off.	PASS
2 · High temperature (25 → 35 °C)	At 3 s the air goes to 35 °C, at 9 s to 28.5 °C (between the two thresholds), at 15 s to 25 °C.	Fan ON and HIGH TEMPERATURE alarm above 30 °C; at 28.5 °C the alarm clears but the fan stays ON (hysteresis); below 27 °C the fan stops.	PASS
3 · Water low	11:59:55 — at 2 s the reservoir drops to 10 %; the 12:00 irrigation comes 3 s later.	LOW WATER alarm, alarm lamp ON, "IRRIGATION skipped: LOW WATER": the pump never runs.	PASS
4 · Irrigation at 12:00	Start at 11:59:56 and run 25 s.	"IRRIGATION start", the pump runs about 5 s (current measured every second), "IRRIGATION end", and only once in that hour.	PASS
5 · Dosing A then B	At 2 s press the DOSING button for 1.5 s.	DOSING A 10 ml (5 s), then DOSING B 5 ml (2.5 s), never together, then DOSING DONE and back to AUTO. Volumes come from the farm model of the pumps (2 ml/s).	PASS
6 · Test mode	21:00 (light off by schedule): at 2 s press the TEST button for 1.5 s.	TEST FAN, TEST LIGHT, TEST DOSE A, TEST DOSE B, TEST PUMP (only because the water is OK), TEST DONE; every load runs once, then AUTO switches the light off again.	PASS
7 · Sensor failure	At 3 s the air sensor (DHT22) gets the fault "Sensor disconnected"; at 10 s the fault is cleared.	SENSOR ERROR alarm, temp=None/null in STATUS, the fan runs (safe side) while the reading is missing; the alarm clears after repair.	PASS
8 · Lighting schedule	Start at 19:59:52 and run 15 s.	The grow light is ON before 20:00 and switches OFF after 20:00 (LIGHT_OFF_HOUR).	PASS
9 · Wiring error (pump wire loose)	The wire from relay contact NO1 to the ACS712 is removed, then the 12:00 irrigation starts (start 11:59:56).	The relay clicks but no current flows: "PUMP FAILURE: no current", PUMP FAILURE alarm, the relay is switched off at once.	PASS
10 · Full system	11:59:52: irrigation at 12:00, at 14 s the air reaches 35 °C, at 18 s press the DOSING button for 1.5 s, at 32 s the air is back to 25 °C.	Irrigation (≈5 s), fan + HIGH TEMPERATURE alarm, dosing A then B, then every alarm clears and the fan stops: the system returns to normal AUTO.	PASS

8. Change One Thing — exercises

1. Change the fan threshold

   Set MAX_TEMP to 28 and FAN_OFF_TEMP to 25. Run, then set the farm air temperature to 29 °C.

   You should see: The fan starts at 29 °C now and the HIGH TEMPERATURE alarm appears.

2. Change the irrigation time

   Set PUMP_SECONDS to 8 and IRRIGATION_HOURS to include the current hour. Run and watch the pump.

   You should see: IRRIGATION start … IRRIGATION end, about 8 s apart; the farm reservoir flow shows the pump running.

3. Move the fan to another GPIO

   Move the wire from GPIO27 to GPIO21 and change Pin(27, …) to Pin(21, …). Do not use GPIO34–39 (input-only).

   You should see: The fan works again from the new pin. If only the code or only the wire changes, the fan stays off while the code thinks it is on (GPIO27).

4. Disconnect the level sensor

   While the program runs, delete the signal wire of the water-level sensor.

   You should see: The level reads wrong (near 0): LOW WATER alarm, the lamp lights, irrigation and dosing are refused. Put the wire back (Ctrl+Z).

5. Change the dose

   Set DOSE_A_ML to 20 and DOSE_B_ML to 10, then start DOSING.

   You should see: A runs 10 s and B 5 s; the Smart Farm totals show about 20 ml and 10 ml. Changing FLOW_ML_PER_SEC without changing the pump only changes the time, not the real flow — that is why calibration matters.

6. Break the pump while it runs

   Select the main pump, set its fault to "Pump failure" and start irrigation (put the current hour in IRRIGATION_HOURS).

   You should see: The relay closes but the ACS712 measures no current: PUMP FAILURE: no current, alarm PUMP FAILURE.

7. Change the lighting hours

   Set LIGHT_ON_HOUR to 8 and LIGHT_OFF_HOUR to 18. Use the clock (Fast-forward) to pass 18:00.

   You should see: The grow light switches off at 18:00 instead of 20:00.

8. Tighten the pH range

   Set PH_MAX to 6.0 (the farm solution is about pH 6.2).

   You should see: ALARM OUT OF RANGE and the lamp lights; nothing else changes because OUT OF RANGE only warns.

9. Building it on real hardware

1. Flash MicroPython for ESP32 (esptool.py --chip esp32 write_flash -z 0x1000 ESP32_GENERIC-....bin).
2. Copy the program as main.py with Thonny or mpremote (mpremote cp esp/main.py :main.py).
3. Power the relays, sensors and the Nextion from a separate 5 V supply and join its GND with the ESP32 GND.
4. ESP32 ADC inputs accept at most about 3.1 V with ATTN_11DB: the ACS712 output (2.5 V + 0.185 V/A) stays below that up to about 3 A; use a divider for bigger pumps.
5. Load the Nextion page (text boxes tTemp ... tDay) into the screen with the Nextion Editor (.tft file).
6. Set the ESP32 clock (ntptime over Wi-Fi or an RTC): the schedule uses time.localtime().

10. Libraries to install

- MicroPython firmware (machine, dht, time built in): https://micropython.org/download/ESP32_GENERIC/
- mpremote / Thonny: pip install mpremote
- Nextion Editor (screen design): Windows application from nextion.tech

11. Simulation vs. real device

- MicroPython 1.29 really runs (WebAssembly); the chip peripherals are simulated: ADC values come from the circuit solver, not from a real ADC with its non-linearity.
- The Nextion is a training model of the instruction set; .HMI/.TFT files are not used.
- The simulator clock is set for you; a real ESP32 starts at 2000-01-01 until you set the time.
- No Wi-Fi or web server is used in this program (none is claimed).

These projects are training references; they do not prove which equipment the final uses nor cover every competition module.